# Classification with Spiking Neural Networks

Andres Cureño Ramirez, and Luis Gerardo de la Fraga

Computer Science Department, Cinvestav

## Introduction

Neuromorphic computing will be important in the future, and we can define it as non-von Neumann computers whose structure and functions are inspired by brains and that are composed of neurons and synapses [1]. Programming a neuromorphic computer often entails creating a Spiking Neural Network (SNN), and have been in the focus of the development of novel computing hardware for Artificial Intelligence, partially because the giant SNN of the brain with about 100 billion neurons consumes just 20W [2].

These types of neural networks do not use non-linear activation functions such as the sigmoid, tanh, or ReLU functions used in Artificial Neural Networks (ANN). Instead, they produce time-dependent spikes. This neuron model can handle the generation of spikes, and these models are generally expressed in the form of ordinary differential equations. In Figures 1 and 2, a comparison between an ANN and an SNN is shown.

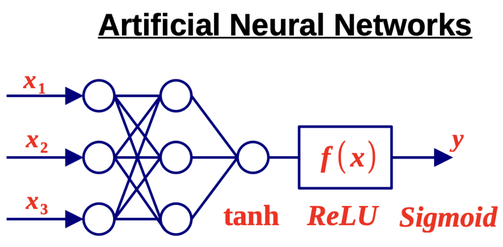



Figure 1: Diagram of an ANN.

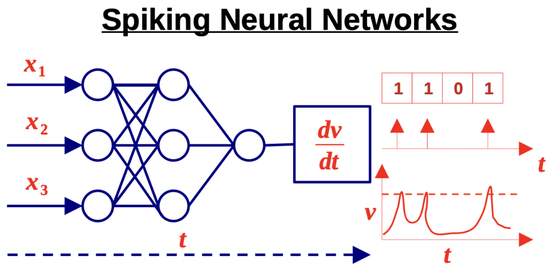



Figure 2: Diagram of an SNN.

There are different models, some are more similar to a biological neuron but they are very computationally expensive and vice versa.

## SNN architecture

We took the proposed architecture for a Recurrent Spiking Neural Network (RSNN) from the work [3] that is described as follows:

$$\begin{aligned} v^{t+1} &= \alpha v^t + W_{\text{rec}} z^t + W_{\text{in}} x^t - z^t v_{\text{th}} \\ z^{t+1} &= H(v^{t+1} - v_{\text{th}}) \\ y^{t+1} &= \kappa y^t + W_{\text{out}} z^{t+1} + b_{\text{out}} \\ s^t &= H(y^{t+1} - y_{\text{th}}) \end{aligned} \tag{1}$$

where $n$ is the number of inputs, $h$ is the size of the recurrent state, $k$ is the number of outputs, and $t$ is the time step. The variables are the input weight matrix $W_{\text{in}} \in \mathbb{R}^{h \times n}$, the recurrent weight matrix $W_{\text{rec}} \in \mathbb{R}^{h \times h}$, the output weight matrix $W_{\text{out}} \in \mathbb{R}^{k \times h}$, the input features $x^t \in \mathbb{R}^n$, the membrane potential of the recurrent state $v^t \in \mathbb{R}^h$, the spikes of the recurrent state $z^t \in \{0,1\}^h$, the biases of the output state $b_{\text{out}} \in \mathbb{R}^k$, the membrane potential of the output state $y^t \in \mathbb{R}^k$, and the spikes of the output state $s^t \in \{0,1\}^k$. The parameters $\alpha$ and $\kappa$ are decay factors, $v_{\text{th}}$ and $y_{\text{th}}$ are the threshold potentials, and $H$ denotes the Heaviside step function. At $t = 0$, the states are initialized to zero ($v^0 = 0, z^0 = 0, y^0 = 0$). When a spike is produced, the corresponding value in $z^t$ or $s^t$ is 1; otherwise, it is 0.

## Results

The predicted class $\hat{c}$ is determined by finding the output neuron that fires the most spikes over the simulation time $T$. This is expressed as:

$$\hat{c} = \underset{i \in \{1,\ldots,k\}}{\operatorname{argmax}} \left( \sum_{t=0}^{T-1} s_i^t \right) \tag{2}$$

where $s_i^t$ is the spike output of neuron $i$ at time step $t$, and $k$ is the total number of output neurons (classes). With the Eq. (2) it is possible to train and evaluate the performance of the Eq. (1). Three datasets were utilized and all the values were normalized between 0 and 1, with 80% of the samples for training and 20% for testing.

The architecture was implemented utilizing PyTorch and SnnTorch [4], which is a framework that has different neuron models implemented. The utilized models were the leaky model [5] and the recurrent leaky model [6], which are based on the Leaky-Integrate-and-Fire model.

### *Circles dataset*

Figure 3 presents the decision space of the model from Eq. (1) with the values from Table 1. Training samples are marked with 'o' and testing samples with 'x'. The model achieved 94.5% accuracy on the test dataset.

Table 1: Hyperparameters of the RSNN for the Circles dataset.

| Samples | $n$ | $h$ | $k$ | $T$ | $\alpha$ | $\kappa$ | $v_{\text{th}}$ | $y_{\text{th}}$ | $b_{\text{out}}$ |
|---|---|---|---|---|---|---|---|---|---|
| 1000 | 2 | 7 | 2 | 5 | 0.95 | 0.95 | 1.0 | 1.0 | 0 |

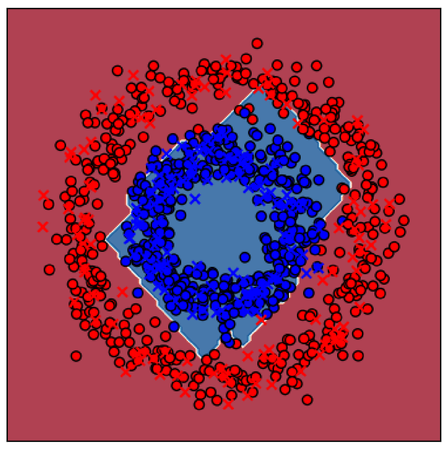


Figure 3: Decision space of the model for circles dataset.

### *Iris dataset*

Figure 4 presents the decision space of the model from Eq. (1) with the values from Table 2. Training samples are marked with 'o' and testing samples with 'x'. The model achieved 100% accuracy on the test dataset.

Table 2: Hyperparameters of the RSNN for the iris dataset.

| Samples | $n$ | $h$ | $k$ | $T$ | $\alpha$ | $\kappa$ | $v_{\text{th}}$ | $y_{\text{th}}$ | $b_{\text{out}}$ |
|---|---|---|---|---|---|---|---|---|---|
| 100 | 2 | 5 | 2 | 5 | 0.95 | 0.95 | 1.0 | 1.0 | 0 |

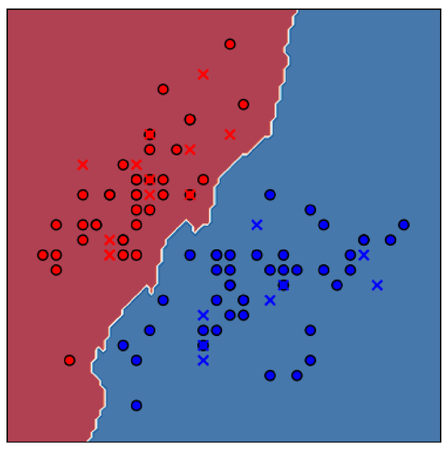


Figure 4: Decision space of the model for iris dataset.

### *Figures dataset*

A dataset of images with eight different figures of 64 × 64 pixels was used [7], and is shown in the Fig. 5.

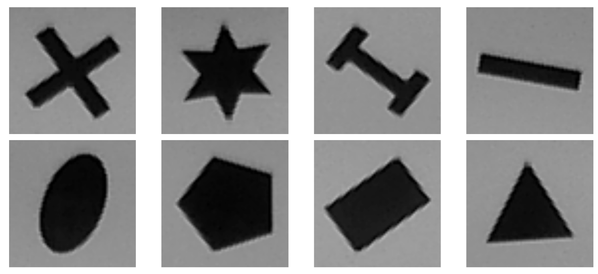


Figure 5: Eight classes of images.

Table 4 presents the confusion matrix of the model from Eq. (1) with the values from Table 3, achieving 99.06% accuracy on the testing dataset.

Table 3: Hyperparameters of the RSNN for the figures dataset.

| Samples | $n$ | $h$ | $k$ | $T$ | $\alpha$ | $\kappa$ | $v_{\text{th}}$ | $y_{\text{th}}$ | $b_{\text{out}}$ |
|---|---|---|---|---|---|---|---|---|---|
| 1600 | 2809 | 50 | 8 | 10 | 0.90 | 0.90 | 1.0 | 1.0 | 0 |

Table 4: Confusion matrix with the testing dataset.

| | | Predicted label | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | Class 1 | Class 2 | Class 3 | Class 4 | Class 5 | Class 6 | Class 7 | Class 8 |
| True label | Class 1 | 50 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| | Class 2 | 0 | 36 | 0 | 0 | 0 | 0 | 0 | 0 |
| | Class 3 | 0 | 0 | 35 | 0 | 0 | 0 | 0 | 0 |
| | Class 4 | 0 | 0 | 0 | 37 | 0 | 0 | 0 | 0 |
| | Class 5 | 0 | 0 | 0 | 0 | 43 | 2 | 0 | 1 |
| | Class 6 | 0 | 0 | 0 | 0 | 0 | 42 | 0 | 0 |
| | Class 7 | 0 | 0 | 0 | 0 | 0 | 0 | 40 | 0 |
| | Class 8 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 34 |

## Conclusions

Even though SNNs do not use non-linear activation functions like ANNs, they can still perform classification tasks by utilizing only the number of spikes from the model's neurons. This makes SNNs much easier to implement in hardware, as it eliminates the need for approximating non-linear functions. An SNN implemented on an FPGA can be considered a hybrid model because it runs on digital hardware. However, these models can also be designed for analog circuits, offering the benefit of lower power consumption.

## References

[1] Schuman, Catherine D and Kulkarni, Shruti R and Parsa, Maryam and Mitchell, J Parker and Kay, Bill and others (2022). *Opportunities for neuromorphic computing algorithms and applications*, Nature Computational Science (pp. 10-19).

[2] Stöckl, Christoph and Maass, Wolfgang (2021). *Optimized spiking neurons can classify images with high accuracy through temporal coding with two spikes*, Nature Machine Intelligence (pp. 230-238).

[3] Bellec, Guillaume and Scherr, Franz and Subramoney, Anand and Hajek, Elias and Salaj, Darjan and Legenstein, Robert and Maass, Wolfgang (2020). *A solution to the learning dilemma for recurrent networks of spiking neurons*, Nature Communications (p. 3625).

[4] snnTorch documentation. https://snntorch.readthedocs.io/en/latest/

[5] snnTorch Leaky Neuron. https://snntorch.readthedocs.io/en/latest/snn.neurons_leaky.html

[6] snnTorch Recurrent Leaky Neuron. https://snntorch.readthedocs.io/en/latest/snn.neurons_rleaky.html

[7] Cureño Ramirez, Andres and García Morgado, Balam and de la Fraga, Luis Gerardo (2024). *Image Classification with Recurrent Spiking Neural Networks*, Mexican Conference on Pattern Recognition (pp. 368-376).